

Snowflake Data Governance Implementation

1. Introduction

This document summarizes the hands-on implementation of **Data Governance in Snowflake**. The objective was to secure sensitive data, enforce controlled access, and ensure data quality using Snowflake's governance features.

The implementation includes:

- Custom role creation (RBAC)
- Tag-based data classification
- Dynamic data masking
- Row-level security
- Data quality checks using DMFs

2. Environment Setup

The following environment was used:

```
USE ROLE useradmin;  
USE DATABASE tb_101;  
USE WAREHOUSE tb_dev_wh;  
  
SHOW ROLES;
```

3. Role-Based Access Control (RBAC)

3.1 Custom Role Creation

A custom role `tb_data_steward` was created to manage governance-related activities.

```
CREATE OR REPLACE ROLE tb_data_steward  
COMMENT = 'Custom Role';
```

3.2 Granting Permissions

Warehouse Access

```
-- Grant warehouse usage
GRANT OPERATE, USAGE ON WAREHOUSE tb_dev_wh TO ROLE tb_data_steward;
```

Database & Schema Access

```
-- Grant database and schema usage
GRANT USAGE ON DATABASE tb_101 TO ROLE tb_data_steward;
GRANT USAGE ON ALL SCHEMAS IN DATABASE tb_101 TO ROLE tb_data_steward;
```

Table-Level Access

```
-- Grant table-level privileges
GRANT SELECT ON ALL TABLES IN SCHEMA raw_customer TO ROLE tb_data_steward;
GRANT ALL ON SCHEMA governance TO ROLE tb_data_steward;
GRANT ALL ON ALL TABLES IN SCHEMA governance TO ROLE tb_data_steward;
```

3.3 Assigning Role to User

```
-- Grant role to your user
SET my_user = CURRENT_USER();
GRANT ROLE tb_data_steward TO USER IDENTIFIER($my_user);
```

3.4 Role Validation

```
-- Switch to the new role
USE ROLE tb_data_steward;

-- Run a test query
SELECT TOP 100 * FROM raw_customer.customer_loyalty;
```

4. Data Classification & Tagging

4.1 Tag Creation

A tag was created to identify **Personally Identifiable Information (PII)** data.

```
CREATE OR REPLACE TAG governance.pii;
GRANT APPLY TAG ON ACCOUNT TO ROLE tb_data_steward;
```

4.2 Enabling Classification Capabilities

Required permissions were granted to use Snowflake's classification features:

```
GRANT EXECUTE AUTO CLASGRANT DATABASE ROLE SNOWFLAKE.CLASSIFICATION_ADMIN TO
ROLE tb_data_steward;
SIFICATION ON SCHEMA raw_customer TO ROLE tb_data_steward;
GRANT CREATE SNOWFLAKE.DATA_PRIVACY.CLASSIFICATION_PROFILE ON SCHEMA
governance TO ROLE tb_data_steward;
```

4.3 Classification Profile Creation

A classification profile was created to automatically detect sensitive data:

```
CREATE OR REPLACE SNOWFLAKE.DATA_PRIVACY.CLASSIFICATION_PROFILE
governance.tb_classification_profile(
  {
    'minimum_object_age_for_classification_days': 0,
    'maximum_classification_validity_days': 30,
    'auto_tag': true
  });
```

4.4 Tag Mapping Configuration

The classification profile was configured to map detected sensitive fields to the PII tag:

```
CALL governance.tb_classification_profile!SET_TAG_MAP(
  {'column_tag_map':[
    {
      'tag_name':'tb_101.governance.pii',
      'tag_value':'pii',
      'semantic_categories':['NAME', 'PHONE_NUMBER', 'POSTAL_CODE',
'DATE_OF_BIRTH', 'CITY', 'EMAIL']
    }
  ]});
```

4.5 Running Classification

```
-- Trigger classification
CALL SYSTEM$CLASSIFY('tb_101.raw_customer.customer_loyalty',
'tb_101.governance.tb_classification_profile');
```

4.6 Verifying Applied Tags

```
-- View applied tags
SELECT
    column_name,
    tag_database,
    tag_schema,
    tag_name,
    tag_value,
    apply_method
FROM
    TABLE( INFORMATION_SCHEMA.TAG_REFERENCES_ALL_COLUMNS('raw_customer.customer_loyalty', 'table'));
```

5. Dynamic Data Masking

5.1 Masking Policy for String Data

```
-- Create the masking policy for sensitive string data
CREATE OR REPLACE MASKING POLICY governance.mask_string_pii AS
(original_value STRING)
RETURNS STRING ->
CASE WHEN
    CURRENT_ROLE() NOT IN ('ACCOUNTADMIN', 'TB_ADMIN')
    THEN '****MASKED****'
    ELSE original_value
END;
```

5.2 Masking Policy for Date Data

```
-- Now create the masking policy for sensitive DATE data
CREATE OR REPLACE MASKING POLICY governance.mask_date_pii AS (original_value
DATE)
RETURNS DATE ->
CASE WHEN
    CURRENT_ROLE() NOT IN ('ACCOUNTADMIN', 'TB_ADMIN')
    THEN DATE_TRUNC('year', original_value)
    ELSE original_value
END;
```

5.3 Attaching Masking Policies to Tag

```
ALTER TAG governance.pii SET
    MASKING POLICY governance.mask_string_pii,
    MASKING POLICY governance.mask_date_pii;
```

5.4 Testing Masking Behavior

```
USE ROLE public;
SELECT TOP 100 * FROM raw_customer.customer_loyalty;

USE ROLE tb_admin;
SELECT TOP 100 * FROM raw_customer.customer_loyalty;
```

6. Row-Level Security (Row Access Policy)

6.1 Creating Role-to-Data Mapping Table

```
CREATE OR REPLACE TABLE governance.row_policy_map
    (role STRING, country_permission STRING);

-- Map the tb_data_engineer role to only see 'United States' data
INSERT INTO governance.row_policy_map
    VALUES('tb_data_engineer', 'United States');
```

6.2 Creating Row Access Policy

```
CREATE OR REPLACE ROW ACCESS POLICY governance.customer_loyalty_policy
    AS (country STRING) RETURNS BOOLEAN ->
    CURRENT_ROLE() IN ('ACCOUNTADMIN', 'SYSADMIN')
    OR EXISTS
    (
        SELECT 1 FROM governance.row_policy_map rp
        WHERE
            UPPER(rp.role) = CURRENT_ROLE()
            AND rp.country_permission = country
    );
```

6.3 Applying Row Access Policy

```
-- Apply the policy
ALTER TABLE raw_customer.customer_loyalty
    ADD ROW ACCESS POLICY governance.customer_loyalty_policy ON (country);
```

6.4 Testing Row-Level Security

```
-- Switch role to test the policy
USE ROLE tb_data_engineer;

-- Query the table
SELECT TOP 100 * FROM raw_customer.customer_loyalty;
```

7. Data Quality Checks (DMF)

7.1 Using Built-in Data Metrics

```
-- This will return the percentage of null customer IDs.
SELECT SNOWFLAKE.CORE.NULL_PERCENT(SELECT customer_id FROM
raw_pos.order_header);

-- We can use DUPLICATE_COUNT to check for duplicate order IDs.
SELECT SNOWFLAKE.CORE.DUPLICATE_COUNT(SELECT order_id FROM
raw_pos.order_header);

-- Average order total amount for all orders.
SELECT SNOWFLAKE.CORE.AVG(SELECT order_total FROM raw_pos.order_header);
```

7.2 Creating Custom Data Metric Function (DMF)

```
CREATE OR REPLACE DATA METRIC FUNCTION governance.invalid_order_total_count(
  order_prices_t table(
    order_total NUMBER,
    unit_price NUMBER,
    quantity INTEGER
  )
)
RETURNS NUMBER
AS
'SELECT COUNT(*)
FROM order_prices_t
WHERE order_total != unit_price * quantity';
```

7.3 Testing Custom DMF

```
-- Insert a record with an incorrect total price
INSERT INTO raw_pos.order_detail
SELECT 904745311, 459520442, 52, null, 0, 2, 5.0, 5.0, null;

-- Call the custom DMF on the order detail table.
SELECT governance.invalid_order_total_count(
  SELECT price, unit_price, quantity FROM raw_pos.order_detail
) AS num_orders_with_incorrect_price;
```

7.4 Automating Data Quality Checks

```
ALTER TABLE raw_pos.order_detail
  SET DATA_METRIC_SCHEDULE = 'TRIGGER_ON_CHANGES';

ALTER TABLE raw_pos.order_detail
  ADD DATA METRIC FUNCTION governance.invalid_order_total_count
  ON (price, unit_price, quantity);
```

8. Conclusion

This implementation successfully demonstrates a **complete Data Governance setup in Snowflake**, covering:

- Role-Based Access Control (RBAC)
- Automated Data Classification & Tagging
- Dynamic Data Masking
- Row-Level Security
- Data Quality Monitoring using DMFs

The solution ensures:

- Secure handling of sensitive data
- Controlled access across roles
- Improved data reliability and trust